

SDN-Lab4

计算机 2303 尤颢辰

2026 年 6 月 2 日

目录

1 Overview of Lab	2
2 WarmUp1: 输出每次影响的等价类数量	2
2.1 Statement of Work	2
2.2 分析在哪里打印这个数量并修改	2
2.3 测试输出	3
3 WarmUp2: 打印环路路径的信息	3
3.1 Statement of Work	3
3.2 分析在哪里打印环路路径并修改	3
3.3 测试输出	4
4 WarmUp3: 打印环路对应等价类的详细信息	4
4.1 Statement of Work	4
4.2 分析代码输出 EC 的函数并修改	5
4.3 测试输出	5
5 实验 1: 修复 Fault 计算	5
5.1 Statement of Work	5
5.2 分析 fault 计数错误原因	5
5.3 修复 fault 计算	6
5.4 测试输出	8
6 选做: 网络验证和分析	10
6.1 Statement of Work	10
6.2 解释产生黑洞的原因	10
6.3 说明这个黑洞会不会影响网络实际使用	10
6.4 修改 as_switch.py	10
7 个人总结	13

1 Overview of Lab

在着手完成实验之前，我们先来看看整个实验的背景如何。

关于网络：实验模拟的是一个跨校共享网络，借鉴了 BGP 自治系统 (AS) 的思想，把网络分成三个 AS。

关于数据包怎么转发：在 AS 内部，直接算最短路径转发；若跨 AS 发送，先在本 AS 内找到通往目标 AS 的边界交换机，边界交换机把包转给它在另一个 AS 的 peer，最终送达。

我们会碰到什么问题：在数据包的传送中，数据包在几个交换机之间绕圈，永远到不了目的地，这就是**转发环路**；数据包到了某个交换机，找不到匹配的流表项，被直接丢弃，这就是**转发黑洞**。

我们期望怎么解决：在这次实验，我们使用 VeriFlow 来解决上面的两个问题。它的核心思想是：每当控制器下发一条新规则，不要等出了问题再排查，而是在规则生效之前就验证它会不会引入故障。

2 WarmUp1: 输出每次影响的等价类数量

2.1 Statement of Work

VeriFlow 验证的核心概念是**等价类 (EC)**。简单来说，一个 EC 就是一组“在整个网络中会被完全相同方式转发”的数据包集合。当控制器下发一条新的流表规则时，不是所有数据包都会受影响，只有一部分数据包的转发路径会因为这条它而改变。我们的第一个热身任务就是：在代码里找到计算“受影响的 EC 的数量”的部分，并把它打印出来。

2.2 分析在哪里打印这个数量并修改

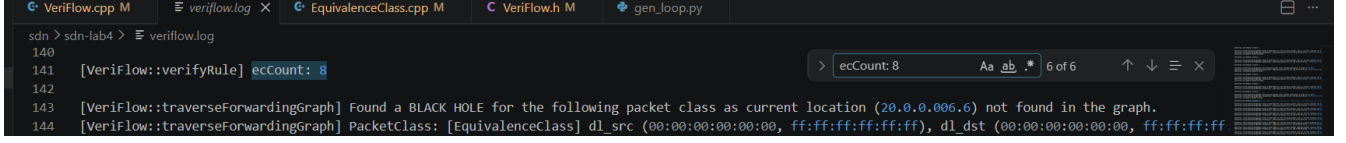
根据实验文档，verifyRule() 是划分等价类的核心函数，EC 数量就应是在这个函数里算出来的。

通读这个函数，我们可以发现这个函数主要实现的任务是：算出受影响的 EC → 为每个 EC 建立转发图 → 遍历转发图查出故障 → 更新故障计数。所以在开头就会实现受影响的 EC 的数量的计算。我们也确实找到了这一部分。代码部分如下：

```
1 bool res = this->getAffectedEquivalenceClasses(rule, command, vFinalPacketClasses, vFinalTries);
2 //计算受影响的等价类，存储到vFinalPacketClasses里
3 .....
4 ecCount = vFinalPacketClasses.size(); //ecCount便是受影响的等价类数量
5 if(ecCount == 0)
6 {
7     fprintf(stderr, "[VeriFlow::verifyRule] Error in rule: %s\n", rule.toString().c_str());
8     fprintf(stderr, "[VeriFlow::verifyRule] Error: (ecCount = vFinalPacketClasses.size() = 0).
9     Terminating process.\n");
10    exit(1);
11 }
12 //原来就有输出部分，我们只需取消注释+改为输出到log中
13 else
14 {
15     fprintf(fp, "\n");
16     fprintf(fp, "[VeriFlow::verifyRule] ecCount: %lu\n", ecCount);
17 }
```

2.3 测试输出

我们开启多个终端。终端 1 用来跑控制器，在 1024 端口监听；终端 2 启动 VeriFlow，工作在 proxy 模式，监听 6633 等交换机来连，收到消息后转发给 1024 端口的控制器；终端 3 运行 mininet；终端 4 用于下发规则制造环路。我们依次运行这几个终端对应的任务，能够截取到 ecCount 的输出：



```
sdn > sdn-lab4 > veriflow.log
140
141 [VeriFlow::verifyRule] ecCount: 8
142
143 [VeriFlow::traverseForwardingGraph] Found a BLACK HOLE for the following packet class as current location (20.0.0.006.6) not found in the graph.
144 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), dl_dst (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff)
```

图 1: ecCount

可以看到，VeriFlow 在 log 中输出了 ecCount=8，说明这条规则导致了 8 个等价类受到了影响。且我们看到了 6 次 ecCount 输出，这恰好对应了 gen_loop 中增加的六条 FlowMod 规则。gen_loop 把这六条规则发给控制器，控制器收到后把每条规则发给对应的交换机，但是被 VeriFlow 截取，随后 VeriFlow 对这些 FlowMod 计算受影响的 EC，被我们输出。

3 WarmUp2: 打印环路路径的信息

3.1 Statement of Work

通过 WarmUp1 我们知道，VeriFlow 可以检测到环路，并输出受影响了的 EC 数量。但光知道“有环路”还不够，我们还需要知道环路长什么样。这就是 traverseForwardingGraph 做的事情。在 verifyRule 之中，对每个受影响的 EC，VeriFlow 构建一张转发图，然后调用 traverseForwardingGraph 去遍历它。我们 WarmUp2 需要做的，就是：修改这个函数，当检测到环路时，把从起点到环路回到起点的完整路径打印出来。

3.2 分析在哪里打印环路路径并修改

经过上面的分析，我们已经知道要在 traverseForwardingGraph 之中去打印环路路径了，下面我们分析分析这个函数：整体上这是一个递归的 DFS 遍历，从当前交换机出发沿着图转移，每走一步递归调用自己。如果再一次走到自己的话就说明碰到环路了。具体的判断环路的代码如下：

```
1 if(visited.find(currentLocation) != visited.end())
2 {
3     // Found a loop.
4     fprintf(fp, "\n");
5     fprintf(fp, "[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node %s.\n", currentLocation.c_str());
6     fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());
7     for(unsigned int i = 0; i < faults.size(); i++) {
8         if (packetClass.subsumes(faults[i])) {
9             faults.erase(faults.begin() + i);
10            i--;
11        }
12    }
13    faults.push_back(packetClass);
```

```

14
15     return false;
16 }

```

但是我们发现了问题，源代码只打印了在哪个节点发现环路以及 EC 信息，但是没有输出路径，且没有数据结构能记录下来路径。所以我们需要自己维护一个结构来记录路径。在这里我选择一个存储 String 的 Vector 来实现这一点。具体要改下面的几点：

1. 在 VeriFlow.h 之中修改函数的声明，改为：

```

1 bool traverseForwardingGraph(..., FILE* fp, vector<string> path = {});

```

2. 修改 VeriFlow.cpp 中的函数签名，和上面的修改方式一样；

3. 修改函数体的逻辑。首先要在遍历到的时候 `path.push_back(currentLocation)`；其次在检测到环路的地方，遍历输出 path；最后修改递归调用时传递的值。

3.3 测试输出

现在我们修改完成了源码，测试能否输出环路到 log 中。还是和 WarmUp1 一样的测试方法，结果如下：

```

sdn > sdn-lab4 > veriflow.log
157
158 [VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
159 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
160 20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
161
162 [VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
163 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
164 20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
165
166 [VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
167 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
168 20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
169
170 [VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
171 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
172 20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
173
174 [VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
175 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
176 20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6

```

图 2: Output(Loop)

我们可以看到有八条环路输出，因为这条新规则影响到了八个 EC。

4 WarmUp3: 打印环路对应等价类的详细信息

4.1 Statement of Work

WarmUp3 要求我们进一步打印出环路对应的 EC 的相关信息。其实我们上一步观察 log 文件的时候，可以看到已经输出等价类的相关信息了，只是输出与我们期望的格式有些区别。所以我们要修改输出 EC 的函数，使输出符合我们的需求。

4.2 分析代码输出 EC 的函数并修改

在输出环路的函数部分，我们看到打印 EC 信息的部分是 EquivalenceClass 类之中的 toString() 方法。下面对它进行修改：仅展示源 MAC、目的 MAC 和 TCP/IP 五元组（即源 IP、目的 IP、协议、源端口、目的端口）。

首先，我们需要去除 dl_src、dl_dst、nw_src、nw_dst 输出的原始数字；

其次，把循环输出所有元素的循环删去，变为只输出 nw_proto、tp_src、tp_dst。

4.3 测试输出

还是同样的测试输出的方法，现在的一条输出被精简为：

```
1 [VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), dl_dst (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), nw_src (10.10.0.5, 255.255.255.255), nw_dst (10.10.0.0, 10.10.255.255), nw_proto (0, 255), tp_src (0, 65535), tp_dst (0, 65535)
```

测试截图：

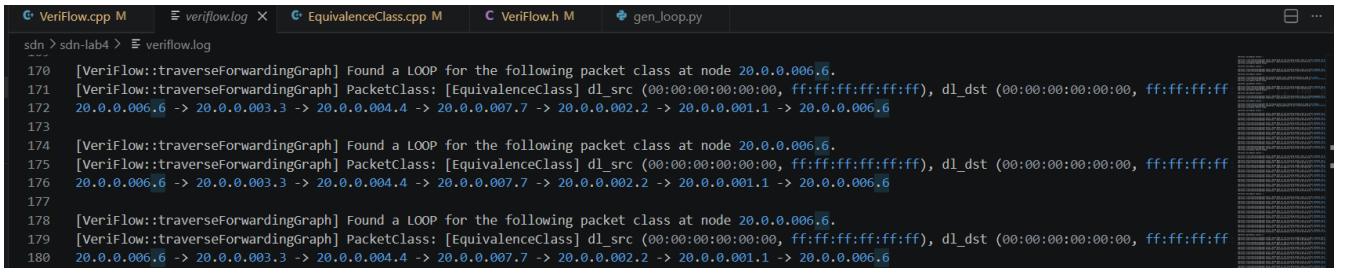


图 3: Result

5 实验 1: 修复 Fault 计算

5.1 Statement of Work

WarmUp 部分结束之后，我们发现 VeriFlow 可以很好地处理环路的部分。现在环路解决了，我们测试跨 AS 的连通性问题，执行 ucla1 ping illinois。结果是能 ping 通的，但是 log 中报了 BLACK HOLE 的出现。

出现黑洞的原因也很好理解。执行 ucla1 ping illinois，控制器下发的流表规则是精确的匹配，即“从 ucla1 去 illinois”怎么走。数据包走到 ucla2 的时候，它上报控制器去 illinois 怎么走，这样控制器指挥下发/32 的匹配。但是当 VeriFlow 检查一个更广的 EC 时，/32 的匹配规则便不足以通过 VeriFlow 的检查，因为其要求等价类里每一个节点都有相同的动作。

可以看到 fix_path.py 给三个交换机补上了/16 子网级别的规则，所以范围大的 EC 也能匹配到相应的规则了。但是问题是：fault 计数并没有消失，我们要解决的就是这个问题。

5.2 分析 fault 计数错误原因

我们简单跑一下，运行 fix_path.py，VeriFlow 的输出如下：

```

veriflow/VeriFlow - sdn-lab4
[sdn@archlinux sdn-lab4]$ veriflow/VeriFlow 6633 127.0.0.1 1024 simple.txt veriflo
w.log
[VeriFlow] [10] Waiting for new connection...
faults size: 1
faults size: 2
Removing fault!
faults size: 1
faults size: 2
faults size: 2
faults size: 2
faults size: 12
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
faults size: 2
faults size: 2

```

图 4: Fault number

可以看到 fault 计数算出来是有问题的，理论上已经应该是 0 了。

下面我们来分析为什么会出现这样的问题。我们假设，faults 里有一个很宽的 fault：

```
1 faults = [Fault_X: dst 10.10.0.0 ~ 10.10.255.255]
```

新规则产生的 EC 比较窄，我们假设是：

```
1 dst 10.10.0.0 ~ 10.10.127.255
```

原有 Fault 计算逻辑只在新的 packetClass 完全包含已有 Fault 时，才会将该 Fault 从 faults 中移除。这个判断过于严格：当新的 packetClass 与已有 Fault 只是部分相交时，新规则实际上已经修复了 Fault 中与 packetClass 重叠的那一部分，但原逻辑会因为 subsumes 判断失败而把整个 Fault 保留下来。

因此，更合理的处理方式是：当 packetClass 与已有 Fault 存在交集时，应当从旧 Fault 中扣除这部分交集；如果扣除后旧 Fault 已经不再覆盖任何数据包，则可以删除该 Fault。也就是说，Fault 的更新不应只判断“是否被完全包含”，还需要考虑“是否被部分修复”。

5.3 修复 fault 计算

分析出问题之后，我们很容易能想到解决的方案：之前的做法是如果 fault 被新划分出来的 EC 的所有维度包含，则把这个 Fault Remove 掉。现在我们需要新写一个 intersect 方法来替代之前的判断逻辑。之前的做法是使用 subsumes 方法来判断，具体如下：

```

1 bool EquivalenceClass::subsumes(const EquivalenceClass &other) const
2 {
3     for(int i = 0; i < ALL_FIELD_INDEX_END_MARKER; i++)
4     {
5         if((this->lowerBound[i] <= other.lowerBound[i] && this->upperBound[i] >= other.upperBound[i]))
6         {
7             //Okay
8         } else {

```

```

9         return false;
10    }
11 }
12 return true;
13 }

```

我们的新方法只需在其基础之上略微改写一下逻辑即可，其能实现判断：是否 EC 和 Fault 在所有维度相交？这样不要求所有维度都被包含，仅需要相交。

```

1 bool EquivalenceClass::intersects(const EquivalenceClass &other) const
2 {
3     for(int i = 0; i < ALL_FIELD_INDEX_END_MARKER; i++)
4     {
5         if(this->lowerBound[i] > other.upperBound[i] || this->upperBound[i] < other.lowerBound[i])
6         {
7             return false; // 不相交
8         }
9     }
10    return true; // 相交
11 }

```

但是之这样修改之后就有新的问题。如果所有维度都有相交部分就会 Remove 掉 Fault，逻辑显然不对。我们的解决方案是：构建一个函数，给它传进去 Fault 与 Packetclass，其能从 Fault 中扣除其与 PacketClass 的交集，并返回新的 Fault 集合。具体实现这个函数的代码如下：

```

1 static vector<EquivalenceClass> subtractEquivalenceClass(
2     const EquivalenceClass& fault,
3     const EquivalenceClass& packetClass)
4 {
5     vector<EquivalenceClass> result;
6     if(!fault.intersects(packetClass))
7     {
8         result.push_back(fault);
9         return result;
10    }
11    EquivalenceClass intersection = fault;
12    for(int d = 0; d < ALL_FIELD_INDEX_END_MARKER; d++)
13    {
14        if(packetClass.lowerBound[d] > intersection.lowerBound[d])
15        {
16            intersection.lowerBound[d] = packetClass.lowerBound[d];
17        }
18
19        if(packetClass.upperBound[d] < intersection.upperBound[d])
20        {
21            intersection.upperBound[d] = packetClass.upperBound[d];
22        }
23    }
24    EquivalenceClass core = fault;
25    for(int d = 0; d < ALL_FIELD_INDEX_END_MARKER; d++)
26    {
27        if(core.lowerBound[d] < intersection.lowerBound[d])
28        {
29            EquivalenceClass left = core;

```

```

30     left.upperBound[d] = intersection.lowerBound[d] - 1;
31     result.push_back(left);
32
33     core.lowerBound[d] = intersection.lowerBound[d];
34 }
35
36 if(core.upperBound[d] > intersection.upperBound[d])
37 {
38     EquivalenceClass right = core;
39     right.lowerBound[d] = intersection.upperBound[d] + 1;
40     result.push_back(right);
41
42     core.upperBound[d] = intersection.upperBound[d];
43 }
44 }
45 return result;
46 }

```

接着，我们还需要在主函数中重写删除 Fault 的循环。要注意，源代码中一共有五处涉及删除 Fault 的循环，前四处负责处理“如果有新的 Fault 覆盖了旧的 Fault”的处理，这个我们不需要管。我们要重写的是后面的“根据新的 EC 来删除可以删除的 Fault”的部分。代码如下：

```

1  for(unsigned int i = 0; i < faults.size(); i++)
2  {
3      vector<EquivalenceClass> remainingFaults =
4          subtractEquivalenceClass(faults[i], packetClass);
5
6      for(unsigned int j = 0; j < remainingFaults.size(); j++)
7      {
8          updatedFaults.push_back(remainingFaults[j]);
9      }
10
11     if(remainingFaults.size() == 0)
12     {
13         fprintf(stderr, "Removing fault!\n");
14     }
15     else if(!(remainingFaults.size() == 1 && remainingFaults[0] == faults[i]))
16     {
17         fprintf(stderr, "Updating fault!\n");
18     }
19 }
20
21 faults = updatedFaults;

```

5.4 测试输出

我们 fix 之后，先执行 illinois ping wisconsin，得到下面的结果：

```
faults size: 1
Removing fault!
faults size: 0
faults size: 1
Removing fault!
faults size: 0
```

图 5: Result

之后执行 `ucla1 ping illinois`, 得到下面的结果:

```
faults size: 1
faults size: 2
Updating fault!
Removing fault!
faults size: 4
faults size: 8
faults size: 8
Updating fault!
faults size: 9
```

图 6: Result

接着用 `fix_path.py` 修复, 得到:

```
faults size: 18
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
faults size: 5
Removing fault!
Removing fault!
Removing fault!
Removing fault!
Removing fault!
faults size: 0
```

图 7: Result

可以看到修复逻辑生效了, 且错误数量和输入错误流表项之前相同。

6 选做：网络验证和分析

6.1 Statement of Work

阅读 `as_switch.py` 有关数据包转发的代码，解释产生黑洞的原因。请说明，产生的黑洞问题会不会影响网络的实际使用？修改 `as_switch.py`，使得同样的 ping 操作不会再产生黑洞。

6.2 解释产生黑洞的原因

之前我们观察到了产生黑洞的原因，下面我们来解释一下：

`as_switch.py` 采用分段式的流表下发方式：当数据包首先在 `ucla1` 触发 `PacketIn` 时，控制器只根据当前交换机所处 AS 和目的网段安装当前这一段路径，例如先安装 `ucla1` 到其 peer `ucla2` 的转发规则。数据包到达 `ucla2` 后，若没有已有匹配规则，再次触发 `PacketIn`，控制器才继续安装 `ucla2` 到 `illinois` 的后续规则。

这种机制对实际 ping 不一定有影响，因为控制器可以在数据包逐跳到达时继续补充流表。但 VeriFlow 检查的是当前已经存在的数据的规则，并且检查对象是一个较宽的 EC，而不是单个 ICMP 数据包。因此，当 VeriFlow 看到某个 EC 已经被转发到 `ucla2`，却在 `ucla2` 找不到后续规则时，就会报告 BLACK HOLE。

`fix_path.py` 的作用就是手动补齐这些缺失的后续转发规则，使 EC 在后续交换机上也能继续匹配到正确流表。这样，原先在 VeriFlow 看来中途断开的 EC 就可以完整到达目的地，Fault 数量也随之恢复。

6.3 说明这个黑洞会不会影响网络实际使用

该黑洞不会影响本次 ping 的实际连通性。由于控制器采用分段的下发规则，数据包每到达一个尚未安装规则的交换机时，都可以再次触发 `PacketIn`，由控制器继续安装后续转发规则。因此，实际的 `ucla1` ping `illinois` 可以成功。

但是，该黑洞反映了当前流表并不是端到端完整的。在控制器尚未补齐后续规则之前，某些属于同一 EC 的数据包如果直接进入中间交换机，或者控制器未能及时响应，就可能没有匹配的下一跳规则。因此，从 VeriFlow 的角度看，它是一个真实存在的潜在黑洞，而不是完全无意义的误报。

6.4 修改 `as_switch.py`

我们需要修改控制器下发流表的逻辑。修改后，控制器在处理跨 AS 数据包时不再只安装当前一跳或当前一段路径，而是先在当前 AS 内安装到 gateway 的最短路径，再安装 gateway 到 peer 的跨 AS 规则，进入下一个 AS 后继续安装后续路径，直到目的 AS 内安装到目的主机的路径。这样首次 `PacketIn` 就能补齐端到端路径，避免 VeriFlow 看到中间断裂的 EC。

为了解决这个问题，我们对 `as_switch` 做以下修改：

首先，添加三个辅助函数：

```
1 # 找某个 IP 实际属于哪个 AS 前缀
2 def find_host_net(ip):
3     for net in self.routing_cfg["gateways"]:
4         if utils.ipv4.in_net(net, ip):
5             return net
6     return None
7 # 在当前 AS 的 gateways 表里找去目的 IP 应该走哪个目标前缀。
```

```

8 def find_route_entry(current_net, dst_ip):
9     if current_net not in self.routing_cfg["gateways"]:
10         return None, None
11
12     for dst_candidate, gateways in self.routing_cfg["gateways"][current_net].items():
13         if utils.ipv4.in_net(dst_candidate, dst_ip):
14             return dst_candidate, gateways
15
16     return None, None
17 # 选择当前交换机到哪个 gateway 最近
18 def closest_gateway(current_dpid, gateways):
19     if current_dpid in gateways:
20         return current_dpid
21
22     min_delay = math.inf
23     min_gw = None
24
25     for gw in gateways:
26         delay = self.network_awareness.shortest_path_length(current_dpid, gw)
27         if delay < 0:
28             continue
29
30         if delay < min_delay:
31             min_delay = delay
32             min_gw = gw
33
34     return min_gw

```

之后，再新增完整的路径安装函数。它会循环安装完整跨 AS 路径。这样就不用等数据包到达每个中间交换机后再次触发 PacketIn。

```

1 def add_complete_path(start_dpid, srcnet, dst_ip, match_dstnet):
2     current = start_dpid
3     first_out_port = None
4     visited = set()
5
6     while True:
7         if current in visited:
8             return first_out_port
9
10        visited.add(current)
11        current_net = self.routing_cfg["switch_nets"][current]
12
13        # 已经进入目的 AS: 安装当前交换机到目的主机的 AS 内路径
14        if utils.ipv4.in_net(current_net, dst_ip):
15            dpid_path = self.network_awareness.shortest_path(current, dst_ip)
16            if not dpid_path:
17                return first_out_port
18
19            out_port = add_path(dpid_path, None, None, srcnet, match_dstnet)
20            if first_out_port is None:
21                first_out_port = out_port
22
23        return first_out_port
24

```

```

25     # 还没进入目的 AS: 先找到当前 AS 到目的方向的 gateway
26     route_dstnet, gateways = find_route_entry(current_net, dst_ip)
27     if route_dstnet is None or gateways is None:
28         return first_out_port
29
30     gw = closest_gateway(current, gateways)
31     if gw is None:
32         return first_out_port
33
34     # 如果当前交换机不是 gateway, 先安装 current -> gateway 的 AS 内路径
35     if current != gw:
36         dpid_path = self.network_awareness.shortest_path(current, gw)
37         if not dpid_path:
38             return first_out_port
39
40         out_port = add_path(dpid_path, None, None, srcnet, match_dstnet)
41         if first_out_port is None:
42             first_out_port = out_port
43
44     # 安装 gateway -> peer 的跨 AS 规则
45     peer = self.routing_cfg["peers"][str(gw)][route_dstnet]
46     out_port = add_path([gw, peer], None, None, srcnet, match_dstnet)
47     if first_out_port is None:
48         first_out_port = out_port
49
50     current = peer

```

最后, 我们还要考虑合适的流表下发顺序。虽然控制器已经能够在第一次 PacketIn 时计算出完整路径, 但这些规则并不是同时安装到所有交换机上的。因此如果仍然按照从源到目的的顺序安装规则, VeriFlow 仍可能观察到中间状态下的黑洞。

因此, 我们采用从目的端到源端的倒序安装策略。对于同一条完整路径, 控制器先安装靠近目的端的规则。代码如下:

```

1  if gateways is None:
2      # 目的地址在当前 AS 内, 保持原来的 AS 内最短路径转发逻辑。
3      # 这里也使用倒序下发, 避免多跳 AS 内路径产生瞬时黑洞。
4      dpid_path = self.network_awareness.shortest_path(dpid, dst_ip)
5      if not dpid_path:
6          return
7
8      out_port = add_path(dpid_path, None, None, src_ip, dst_ip)
9  else:
10     # 目的地址不在当前 AS 内。
11     # 第一次处理数据包时先收集完整跨 AS 路径, 再从目的端往源端安装流表。
12     out_port = add_complete_path(dpid, srcnet, dst_ip, match_dstnet)
13     if out_port is None:
14         return

```

这样, 我们就解决了黑洞的问题。下面我们执行 ucla1 ping illinois 验证一下:

```
faults size: 0
faults size: 0
faults size: 0
faults size: 0
faults size: 0
faults size: 0
```

图 8: Result

对比之前的结果可以看到，在这个过程中没有出现 Fault！这代表我们的修改是正确的。

7 个人总结

至此，本学期的 SDN 实验任务就结束了，自我总结下来还是学到了不少东西的，所以想在最后留一段简短的总结。我认为这种布置较难以实现的 Lab，而非公式化的考试的学习模式是一种相当有效率且能学到东西的模式。在学习这门课之前，我很少使用 python 语言编程，也对其基本语法了解甚少。但是现在不说能熟练编写 python 脚本，读懂且熟悉基本语法是完全没有问题的。在此之余我对网络知识也有了很大的了解，事实上网络并不是只要“连起来”就结束了，真正重要的是理解数据包为什么会这样走、规则为什么会这样生效。从 Mininet 和 OVS 的基础操作，到 BGP、交换机，再到最后的 VeriFlow 验证，我逐渐体会到 SDN 的核心价值在哪里，其与基本的网络知识有什么不同与优势。虽然实验过程中反复的调试给我带来了不少不好的记忆，但这些过程也让我切实学到了不少知识！